

PRD: UX Audit Agency

1. Problem Statement

Small product teams and solo builders ship interfaces without a UX expert in the room. They know something feels off — users drop, forms get abandoned, pages underperform — but they lack the vocabulary or trained eye to diagnose why. Professional UX audits exist but are expensive (often thousands of dollars), slow (days to weeks), and inconsistent in scope. Rigorous, structured UX critique is locked behind specialists, so the people who most need it get none — and problems only surface in churn or support tickets, when the fix is already costly.

2. Goals

1. Deliver a full structured UX audit in under 2 minutes from input to report.
 2. Achieve 90%+ routing accuracy — the orchestrator spawns the correct set of sub-agents for a given input on a test set of varied pages.
 3. Achieve 80%+ precision on findings — issues surfaced are real and actionable, not false positives, scored against an expert rubric.
 4. Cover 8 of 10 deliberately seeded issues on a test page (coverage rate).
 5. Demonstrate strong severity ranking quality — synthesizer's top-5 list matches expert-ranked priority at a statistically meaningful rank correlation.
-

3. Non-Goals

- No automatic code fixes, patches, or pull requests — the system diagnoses, it does not implement.
 - No visual redesigns or design mockups.
 - No native mobile app auditing — web only for v1.
 - No continuous monitoring or scheduled re-audits.
 - No integration with user behavior analytics or session replay tools.
 - No auditing of auth-walled or logged-in flows — public pages and uploaded screenshots only.
 - No multi-language or localization review.
-

4. Users & Use Cases

Scenario 1 — The Indie SaaS Founder Marcus built a SaaS tool for freelancers and just launched. Signups are decent but trial-to-paid conversion is low. He suspects the onboarding flow is confusing but has no designer. He pastes his app URL into the audit tool, gets a prioritized report within two minutes, and sees that three onboarding steps violate consistency heuristics and his CTA copy is ambiguous. He fixes them before his next cohort.

Scenario 2 — The No-Code Builder Priya built a client portal using Webflow and Notion. A client mentioned it "feels clunky" but couldn't say more. She uploads screenshots of the key screens. The tool surfaces contrast failures, a form with missing labels, and microcopy that creates uncertainty at the submit step. She has concrete fixes to give her developer by end of day.

Scenario 3 — The Generalist PM James owns the checkout flow at a 30-person startup. Design is booked out three weeks. He needs a fast first-pass before his next sprint planning. He runs the audit, shares the structured findings doc with his team, and uses it to prioritize which issues are worth queueing design time for versus quick dev fixes.

5. User Stories

Must-Have

- As a solo founder, I want to submit a URL so that the system automatically audits my live page without me preparing anything.
- As a builder without design tools, I want to upload screenshots so that I can audit flows I can't share via URL (e.g., staging, client work).
- As a user, I want the orchestrator to automatically detect what's on the page and spawn the right agents so that I get relevant findings, not a generic checklist.
- As a user, I want findings evaluated against Nielsen's 10 heuristics so that I receive structured, expert-grounded feedback.
- As a user, I want a synthesized, deduplicated, severity-ranked list of issues so that I know what to fix first without wading through raw agent output.
- As a user, I want each finding to include location, severity, and a concrete suggested fix so that I can act immediately.

Should-Have

- As a user with a form-heavy page, I want a form/flow agent to activate automatically so that friction points in my critical conversion path are flagged.
- As a user on a public site, I want an accessibility agent to evaluate contrast, tab order, alt text, and focus states so that I catch WCAG failures without a specialist.
- As a developer consuming the output, I want structured JSON findings (issue / location / severity / evidence) so that I can integrate or display results programmatically.

Nice-to-Have

- As a user, I want a shareable HTML report so that I can send findings to a collaborator or client without them needing access to the tool.
- As a user, I want a copy/microcopy agent to flag confusing labels and tone issues so that I improve UX beyond layout and structure.
- As a user, I want a competitor-benchmark agent to compare my patterns against category rivals so that I understand how my UX stacks up in context.
- As a returning user, I want to re-audit after making changes and see a diff so that I can confirm improvements and track what's outstanding.

6. Acceptance Criteria

Given a valid URL is submitted When the orchestrator processes the input, Then it spawns the heuristics agent and synthesizer at minimum, and conditionally spawns accessibility and form agents based on detected page content — within 120 seconds total.

Given a set of screenshots is uploaded When the orchestrator processes the input, Then it identifies distinct screens, routes them to the appropriate agents, and produces findings scoped to each screen.

Given the heuristics agent completes analysis When findings are returned, Then each finding includes: heuristic violated (by name and number), location on page, severity (critical / major / minor), and a suggested fix — no finding is prose-only.

Given the synthesizer receives findings from multiple agents When deduplication runs, Then no single issue appears more than once in the final output, and conflicting severity ratings between agents are resolved by the higher severity.

Given the synthesizer produces a final report When output is rendered, Then the top-5 issues are displayed first, ordered by severity score, with each issue actionable without reading the full report.

Given any input (URL or screenshots) When the audit completes, Then wall-clock time from submission to output is under 120 seconds. [NEEDS INPUT — confirm acceptable p95 latency target]

7. Risks & Assumptions

Risks	Assumptions
Screenshot-only inputs lack DOM data; accessibility and tab-order checks will be incomplete or unavailable	[ASSUMPTION] Screenshot inputs are sufficient for heuristics and copy analysis even without DOM access
LLM hallucination risk — agents may flag issues that don't exist or miss real ones	[ASSUMPTION] GPT-4 Vision or Claude with vision is capable enough to analyze screenshots for heuristics violations without ground-truth DOM
Pages with heavy JS rendering (SPAs) may not be fully captured via URL scrape	[ASSUMPTION] A headless browser (Playwright/Puppeteer) will be used for URL inputs to capture rendered DOM and screenshot
Synthesizer dedup logic may miss semantically equivalent issues phrased differently by different agents	[ASSUMPTION] Each agent returns structured JSON, not prose, making dedup tractable
Rate limits and latency on parallel agent calls could push total time over 2 minutes	[ASSUMPTION] All sub-agents can run in parallel, not sequentially
Users may submit auth-walled pages, expecting results they won't get	[ASSUMPTION] Clear UX on input step communicates the public-only limitation before submission

8. Open Questions

- What model and vision capability will power each agent — Claude with vision, GPT-4o, or a mix? Who decides? [NEEDS INPUT — Hamza to confirm]
- How does the orchestrator detect page type (form-heavy, content, SaaS app) — rule-based on DOM elements, LLM classification, or both? [NEEDS INPUT — architecture decision before sprint 1]
- What is the data retention policy for submitted URLs and screenshots — are they stored, and for how long? [NEEDS INPUT — legal/privacy consideration]
- Is the output exclusively in-browser, or do we need a copy/download option for the HTML report in v1? [NEEDS INPUT — affects frontend scope]
- For the competitor-benchmark agent, is web search a hard dependency, or can it run against a cached set of known patterns? [NEEDS INPUT — scoping decision for nice-to-haves]
- What constitutes the "test set" for routing accuracy and coverage metrics — who builds and maintains it? [NEEDS INPUT — QA/eval ownership]

9. Success Metrics

Leading Indicators (early signals — measurable during build)

- Routing accuracy on a 10-page test set: $\geq 90\%$ correct agent selection
- Coverage rate on a seeded test page: catches ≥ 8 of 10 planted issues
- Median time-to-audit on 5 test pages: under 120 seconds

Lagging Indicators (product outcomes — measurable post-launch)

- Finding precision: $\geq 80\%$ of surfaced issues rated "real and actionable" by expert reviewer
- Severity ranking quality: rank correlation between synthesizer top-5 and expert-ranked top-5 is statistically significant (Spearman $\rho > 0.7$) [ASSUMPTION — threshold is reasonable for v1]
- User activation: % of users who complete an audit and return for a second within 7 days
- Report share rate: % of completed audits where the HTML report is downloaded or shared [NEEDS INPUT — only relevant if HTML report ships in v1]

Review the [ASSUMPTION] and [NEEDS INPUT] sections before sharing with engineering.